Project 2: Model Engineering

Bag of Words (BoW), Feed Forward Neural Network (FFNN),

Recurrent Neural Network (RNN) and Graph Neural Network (GNN)

1 Objectives

In this lab you model **the same data** to explore how *data representation* and *model engineering* affect classification performance. The dataset consists of sequences of Windows API calls of different software, each sequence is labelled as *malware* or *goodware*.

Starting from a **Bag-of-Words (BoW) baseline**, you will move towards different *data representation* and **model engineering** steps: deciding how to represent the same sequence in inputs with different neural network architectures.

The lab follows this progression:

- **Bag-of-Words (BoW):** each API call sequence can be represented through the frequencies of individual calls (*unigrams*) or adjacent pairs of calls (*bigrams*). You will explore how these representations perform with shallow learning classifiers and whether capturing local order through bigrams improves classification.
- **Feed-Forward Neural Network (FFNN):** each API call sequence can be encoded as a fixed-length input to a feed-forward network. You will explore how to encode API calls and handle executions of different lengths, and how these choices affect the information available to the model and its classification performance.
- **Recurrent Neural Network (RNN):** since API calls form a **sequence**, RNNs seem a natural choice for modelling their order. You will explore whether processing calls sequentially improves classification and whether the relevant patterns involve short-term and/or long-term dependencies between calls.
- **Graph Neural Network (GNN):** An API call sequence can also be represented as an **API call graph**: a directed graph describing transitions between consecutive API calls. You will explore whether this representation improves classification, how the choice of nodes, edges, features, and model architecture captures software behaviour, and which aspects of the original sequence are preserved or discarded.

Throughout this progression, you will examine **what each representation preserves and what it discards**. More elaborate models introduce additional design choices and computational costs; your experiments will help to establish when these are justified.

Central question: *Do call order and transition structure matter for this dataset?*

You will compare the approaches and investigate their behaviour through controlled changes to the input, including experiments that alter call order.

As in Lab 1, you will have to make your modelling and experimental choices. We provide recommended starting points; you can revise them as you learn more about the data. Some attempts may not improve performance or may fail to train as expected. Analysing these results is part of the lab and can help you take further decisions.

Your final discussion should explain the choices you made, relate your conclusions to the results, and identify any limitations or open questions.

A note on AI assistants. You may use AI coding assistants to help you implement models, write training loops, or debug your code. However, you are responsible for understanding, checking, and validating the code you use. You must be able to identify and fix bugs, including those in AI-generated code (notice this skill can be assessed as part of the exam). This requires knowing what the implementation should do and being able to recognise errors. You should be able to explain how your code works and why it is appropriate for the experiment you are carrying out.

The lab also requires you to make modelling choices and interpret your results. This includes deciding how to represent the data, considering what information each representation preserves, and understanding why a model performs well or fails to train. AI assistants can support this work, but you need to assess their suggestions in the context of your data and experiments. Unexpected results and unsuccessful attempts are part of the learning process: discuss what you observed, what might explain it, and how it informed your next steps.

2 Lab Rules

Each student must join a self-organized group of 4 students. Each group submits **four zip files**, one per lab, named `groupID_labX.zip`. Each zip contains:

- **A report** (max 10 pages) describing your approach, decisions, experiments, and results, with tables and plots. Name it `groupID_labX.pdf`. The report is judged mainly on *reasoning* (see the rubric), not on raw performance numbers.
- **Your experiment logbook** (a section/appendix of the report or a separate sheet – see Section 4). This is required, and does not count in the number of pages.
- **The notebook(s) and code** you wrote.
  - *Best practice:* use Markdown headers and comments to (i) understand tomorrow what you did today and (ii) make us follow your solution.
  - **The notebook must be executed**: code *and* outputs must be visible.
  - **It must run** if we re-execute it. Set seeds where you rely on reproducibility.
  - **Format:** include the `.ipynb` and an HTML export.

Notice that this report **contributes to your examination** as described on the teaching portal. You might submit this report up to **10 days** before any exam session.

3 Malware vs. goodware from API-call sequences

The data comes from research on behavioural malware detection: programs were run in a Cuckoo Sandbox, and the sequence of Windows **API calls** they made was recorded. Each sample is one such sequence, labelled as produced by *malware* or by benign software ("*goodware*"). Background: Kaggle and Oliveira & Sassi (2019), *Behavioral Malware Detection Using Deep Graph Convolutional Neural Networks*, TechRxiv, https://doi.org/10.36227/techrxiv.10043099.v1.

You are given two JSON files, `train.json` and `test.json` (the split is done for you). Each entry has two fields:

- `api_call_sequence`: a list of **variable-length** API-call names (strings) for example: `["LdrLoadDll", "RegOpenKeyExW", "NtClose", "LdrLoadDll",...]`.

  Consecutive *identical* calls have been collapsed, but the same call can reappear later in the sequence (e.g., **"LdrLoadDll"** above).

- `is_malware`: the label — `0` for goodware, `1` for malware.

The first task is to explore the dataset, examining the class balance, sequence-length distribution, vocabulary size, and differences between the training and test sets. This analysis will help guide your modelling and evaluation choices, including the performance metrics used throughout the lab.

> ***The test set.*** *Use the test set* ***only to evaluate the final performance*** *of your models. Create a* ***validation set*** *from* `train.json` *to guide model selection and hyperparameter tuning, choosing and justifying an appropriate splitting strategy.*

4 Logbook, budget, hypotheses, and ablations

1. The experiment logbook (required deliverable)

Keep one logbook for the whole lab, report **only the training runs**, including BoW classifiers.

Run	Task	What I changed (one thing)	Hypothesis *before* running	Key result (val)	What I concluded
1	2A	baseline: unigram BoW + logistic regression	call frequencies may distinguish the classes, but goodware may be harder	macro-F1 0.xx; goodware recall low	baseline established; minority-class detection needs attention
2	2B	unigrams → unigrams + bigrams; same classifier settings	adjacent call pairs may improve classification	macro-F1 slightly lower; longer training time	no improvement in this run, contrary to my hypothesis

Only runs that train a model go in the logbook. Preprocessing and data-analysis decisions (Task 1) belong in your report text, not here.

- **Write the hypothesis *before* you run.** A wrong hypothesis you then explain is worth *more* than a vague right one. Do not rewrite history.
- **Change one thing per run.** If you change two things at once, you can't tell which one caused the effect.

2. The run budget

You have a budget of ∼**30 logged training runs for the whole lab**. The deep models (RNN, GNN) are heavier than Lab 1, so spend deliberately: before each run, your logbook must say *why this run is worth one of your remaining slots.* A rough guide: a handful for BoW (Task 2), ∼6 for the FFNN (Task 3), ∼8 for the RNN (Task 4), ∼8 for the GNN (Task 5). You do not have to use them all.

> *The budget is the skill, not a hardware limit. In real settings every experiment costs money, time, and attention, and disciplined experimentation is what separates an engineer from someone trying things at random. "I ran a grid over everything" is* ***not*** *a good answer here.*

3. Expected and observed results

Before each experiment, **think about what you expect** to happen **and why**. Use this as a hypothesis to check against the results. You do not need to write down each hypothesis, but discuss any relevant or unexpected findings.

4. Test if the order really matters

Several times in this lab you may claim that a model "uses the order of the API calls". To investigate this claim:

1. Take a trained model and evaluate it on the original test set.
2. Randomly shuffle the calls within each test sequence, keeping the same calls and occurrence counts.
3. Evaluate the same trained model on the shuffled test set, without retraining. Keep the evaluation settings unchanged and compare the results.

Use the following questions to interpret the comparison:

- How does shuffling affect the input representation of each model? For unigram BoW, should the vectors change? Check your expectation against your implementation.
- Does performance improve, degrade, or remain unchanged? What can you conclude about the model's use of call order in each case?
- If performance remains unchanged, is this sufficient to conclude that the model ignores call order? Could individual predictions still change?
- Can this experiment distinguish the contribution of local order from that of longer-range dependencies? Explain why.

Apply this comparison to the BoW, FFNN, RNN, and GNN models. For the FFNN, follow the instructions in Task 3 to keep truncation and padding from introducing additional changes.

Shared setup for the neural models (Tasks 3–5)

You met these tools in Lab 1 / the lectures; reuse them so that a model's failure is a real finding and not a setup artefact:

- **Class imbalance.** In Task 1, you will investigate the class balance and consider how it may affect classification. Based on your findings, decide whether and how to address it, keeping a consistent approach across models. Explain your choice of **loss function** and, if you use class weights, how you compute them and which data partition you use to estimate them.
- **Vocabulary and unseen calls.** Build the API-call vocabulary using only the training partition, excluding validation and test data. Explain how you handle calls that are absent from this vocabulary. For sequence models, also explain how padding is encoded and distinguished from actual API calls.
- **Training sequence models.** Long sequences can make Recurrent Neural Networks training unstable. You can adjust the learning rate or vary the LSTM cells. Consider that the training instability can be due to large gradients, gradient clipping may help.[1] If you use padding, make sure the model does not treat it as real API calls.

5 Task 1 – Look at the data before you model it

Inspect the data and report what you found and what it implies.

1. **Class balance.** Report the proportion of malware vs goodware in train and in test.
   - What *accuracy* would a classifier achieve if it **always predicted the majority class**? Based on this result, why can accuracy be misleading for this dataset?
   - **Q:** Choose **which metric(s)** to report for the rest of the lab and explain why. Note that goodware is the **minority class**. For a real malware detector, what are the consequences of **missing malware (false negative)** compared with **classifying goodware as malware (false positive)**? There is no correct answer – use these considerations to support your choice.
2. **Sequence lengths.** Plot the distribution of sequence lengths *separately for train and test*, and compare them. You can use the full content of *train.json* without splitting and *test.json*.
   - **Q:** Are the two distributions similar? Report the minimum, maximum, and typical sequence length for each, and describe any differences. Consider how these differences might affect the models you will use in Task 3.

   Vocabulary. For this analysis, use the training partition obtained after creating the validation split. Count its distinct API calls.
   - **Q:** Are there API calls in the test set that do not appear in this training partition (out-of-vocabulary calls)? How many distinct calls are there, and how many test samplescontain them? In Task 2, you will consider how your BoW representation handles these calls.

[1] In PyTorch, use `torch.nn.utils.clip_grad_norm_` after `loss.backward()` and before `optimizer.step()`. See https://docs.pytorch.org/docs/2.14/generated/torch.nn.utils.clip_grad_norm_.html.

6 Task 2 – The Bag-of-Words (your baseline to beat)

Treat each sequence as a "sentence" whose "words" are API calls, and fit an `sklearn` `CountVectorizer` to solve the first BoW objective. Fit the vectorizer using only the training partition, excluding validation and test data. Use the fitted vectorizer to transform the validation and test sequences.

6.1 2A – Unigram BoW (order-blind)

Represent each sequence using the frequencies of individual API calls (*unigrams*) and train a classifier of your choice, such as a shallow learning model or an FFNN with **few layers**. Tune and explain its hyperparameters and address any class imbalance identified in Task 1.

- **Q:** How many columns does the vectorizer produce, and why does it produce that number? What does each row represent? Can you recover the original call order from this representation?
- **Q:** Give an example of **two different API call sequences that produce the same unigram vector**. What information is lost in this representation? Can two sequences containing the same API calls still produce different unigram vectors? Give an example and explain why.
- **Q:** Out-of-vocabulary: what does `CountVectorizer` do at test time with the calls you counted in Task 1 that it never saw in training? Does it raise an error, ignore them, or handle them in another way? How many test samples does that affect?
- **Predict first:** will an order-blind model do well *on your chosen metric* (not accuracy)? Train, and report. Explain the per-class result – in particular, what happens on the rare class.

6.2 2B – Bigram BoW (local order)

Now include both individual API calls and adjacent pairs of calls. Configure `CountVectorizer` with `ngram_range=(1, 2)`. We refer to this combined representation as the **bigram BoW model**.

- Do you expect the model to perform better when it also knows which API calls appear next to each other, rather than only how often each call appears? Think about why and motivate it. Then train the model and compare the results with Task 2A.
- **Q:** Did the results match your expectations? If bigrams improved classification, why might pairs of consecutive API calls help distinguish malware from goodware? If there was little or no improvement, what might explain this?
- Apply the shuffle test described in Section 4. Check that shuffling the calls leaves the *unigram* vectors unchanged but can change the *bigram* vectors. Briefly explain why. Compare the performance of the *unigram* and *bigram* models on the original test set and on the same test set after shuffling the API calls within each sequence. Does performance change for either model? Explain the results.

*Use the **best BoW model** (unigram or bigram), selected using the validation set, as the **baseline** for the rest of the lab. Report the selected performance metric in the validation set and compare the FFNN, RNN, and GNN results with this baseline.*

7 Task 3 – FFNN: forcing sequences into fixed windows

A feed-forward network takes a fixed-size input vector.

- **Q:** Why can an FFNN *not* consume a variable-length sequence directly? How will you use padding and/or truncation to give every input the same length?
- **Choosing the sequence length.** Choose a fixed length L using your findings from Task 1. If L is the length of the longest training sequence, how will you handle longer sequences at test time? Are there any in your dataset? Explain your choice and its consequences.

Build **two FFNNs** with the same architecture, **changing only** how you encode the API calls and adjusting the input layer accordingly. Use the following encoding methods:

- **(a) Integer IDs:** assign each API call a unique integer ID and feed the sequence of IDs directly into the network, applying scaling if appropriate.
- **(b) Learnable embeddings:** map each API call to a trainable embedding using `nn.Embedding`, then flatten the sequence of vectors and feed it into the network.

Now answer the following questions:

- Before training: Integer IDs assign a numerical order to API calls. Is it meaningful? Which encoding do you expect to achieve better performance, and why?
- Train both models. **Q:** Do they achieve similar performance? Does one take longer to train or show less stable training? Use your training and validation curves to explain the results.
- **Q:** Compare your **best FFNN** with the BoW baseline from Task 2. Does keeping call positions in a fixed-length sequence improve classification? What might explain the results?
- **Q:** Select your best FFNN architecture and compare its average training time per epoch on CPU and GPU, using the same model configuration and batch size. How much faster or slower is training on the GPU?
- **Order ablation.** Evaluate your best FFNN on the original and shuffled test inputs. If you truncate a sequence, first select the calls retained in the input, then shuffle only those calls. Apply padding afterwards, keeping its positions unchanged between the two versions. **Q:** Compare performance before and after shuffling. What does the result suggest about the model's use of call positions?

8 Task 4 – RNN: full order, any length

Now use an RNN to process each sequence.

- **Q:** Do you still need to **pad** or **truncate** sequences in the training/validation set? Do you need to **truncate test sequences** that are longer than those used for training? Explain your answers based on how an RNN processes a sequence.
- **Q:** Compare the number of trainable parameters in your RNN and FFNN. If you increased the input sequence length, would the number of parameters increase in either model? Explain why.

Train **three variants** and compare their performance and training behaviour. Use the results of the first model, a **unidirectional vanilla RNN**, to guide your design choices for the next two architectures.

- **Unidirectional vanilla RNN.** Train the model and plot the training and validation loss curves. If you observe training difficulties, explain the possible causes.
- **Bidirectional RNN. Q:** Is using information from later calls appropriate for this task? Consider whether detection is performed during execution or after the full sequence has been collected.
- **LSTM.** Choose a unidirectional or bidirectional model and explain your choice. Compare its results and training curves with the other two variants.

Then:

- **Q:** Compare the training time per epoch of the RNN and FFNN. Which is faster, and what explains the difference?
- **Q:** Select your best RNN architecture and compare its training time per epoch on the CPU and GPU. How much faster or slower is it on the GPU?
- **Order ablation.** Evaluate your best RNN on the original and shuffled test sets. **Q:** How does performance change? What does this suggest about whether the model uses call order?
- **Q:** Compare your best RNN with the FFNN and BoW models, including the **bigram baseline**. Does processing the calls sequentially improve classification? Discuss possible reasons for the results.

9 Task 5 – GNN: the sequence as a graph

Finally, represent each API call sequence as a graph and train a GNN to classify it.

Before training:

- **Graph construction.** Choose your nodes, edges, and node features, and explain your choices. For example, you could use one node per distinct API call and a directed edge $a \to b$ whenever call a is immediately followed by call b.
  - **Q:** What information does your graph preserve, and what does it lose? Can you recover the original call order and the number of times each call appears?
  - **Q:** What would self-loops and cycles represent? Consider that consecutive repeated calls were removed from the sequences, but the same call can appear again later.
  - **Q:** What node features will you use? For example, you could assign a learnable embedding to each API call.

- **Graph visualization.** Visualize the graph constructed from one sample API call sequence. Include the original sequence and label the nodes and edges to make the conversion clear[2].
- **Q:** Do you need **padding or truncation** when using a GNN? Explain your answer based on how a GNN processes a graph, and compare this with the fixed-length input required by your FFNN in Task 3.

Now build and train:

- **GCN, GraphSAGE, and GAT.** Consider how their message-passing and aggregation methods differ, then train the models and compare their validation performance. Keep the other settings as similar as possible to help interpret the differences.
- **Order ablation.** Build graphs from the shuffled test sequences and evaluate your best GNN on them. **Q:** Compare its performance with that on the original test graphs. What does the result suggest about whether the model uses the transitions between calls?
- **Q:** Compare your best GNN with the BoW, FFNN, and RNN models. Does the graph representation improve classification? Discuss possible reasons for the results.

[2]You can use `NetworkX` to do that.

10 Task 6 – Verdict: where do the gains stop?

Compare all approaches on the test set using the metrics chosen in Task 1. For each approach, use the model selected on the validation set.

Report its test score before and after shuffling, and the total training time of the selected run, excluding hyperparameter search. State the device used and, for neural models, the number of training epochs. Explain whether any performance improvement justifies the additional training time and complexity.

Model	**Test score**	**Training time**	**Test score after shuffling**	**Worth the cost?**
Unigram BoW				
Bigram BoW				
Best FFNN				
Best RNN				
Best GNN				

- **Q:** Which approaches improved performance over the simpler models? When did additional complexity provide little or no improvement?
- **Q:** What do your results suggest about the importance of call frequencies, local order, and longer-range dependencies for this task? Support your answer with your comparisons and shuffle-test results, considering what these experiments can and cannot establish.
- **Q:** Were the improvements worth the additional training time and complexity? If BoW performed similarly to the neural models, discuss possible reasons. If a neural model performed better, what evidence helps explain its advantage?

11 Self-assessment Evaluation Process

Lab 2 is graded. Use the following points to check your report and code before submission. ***We value clear reasoning over high performance and do not compare results across students.*** *You must understand your code and be able to identify and fix bugs. We assess* ***YOUR understanding of the subject – not that of an LLM.***

Criterion	**What you should provide**
Representation choices	Explain how you represent the API call sequences for each model, what information is preserved or lost, and how these choices may affect classification. Justify decisions such as the sequence length, call encoding, graph construction, and node features.
Metrics & class imbalance	Explain your choice of evaluation metrics, considering the class balance and the consequences of false negatives and false positives. Describe whether and how you address imbalance, including how any class weights are computed and which partition is used to estimate them.
Experimental approach	Use your expectations to guide experiments and compare them with the observed results. Explain the main changes you tried and what you learned from them, including unsuccessful attempts. You do not need to write down a hypothesis for every run.
Effect of call order	Compare performance on the original and shuffled test sets using the same trained models. Check that unigram representations remain unchanged and explain how shuffling affects the other representations. Discuss what the results suggest about the models' use of call order.
Interpretation of results	Support your explanations with your own curves, measurements, and results. Consider relevant observations such as differences in sequence length, unseen API calls, training difficulties, and performance changes after shuffling. Discuss possible causes without assuming that a particular model must succeed or fail.
Comparison & complexity	Compare the FFNN, RNN, and GNN with the BoW baseline. Consider classification performance, training time, and model complexity, and explain whether the more elaborate approaches were useful for this dataset.
Code & reproducibility	Provide code that runs correctly and that you can explain, check, and debug. Keep training, validation, and test data separate in your experimental process. Document the settings needed to reproduce your results, including random seeds where relevant.
Report clarity	Provide a clear report within the page limit. Connect your conclusions to the experiments you performed and explain any limitations or unresolved questions.